FRED TALKS

27th May 2026

CONTENTS

Finding Vulnerabilities with Warden

CRA.MR · 28 APR 2026 · SOURCE

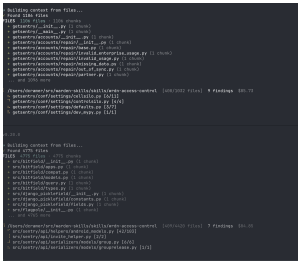
We found more than 100 previously unknown vulnerabilities across Sentry and a few other open source projects with Warden.

I'm here once again to convince you this is worth your time, and to explain our approach.

What is Warden

Warden is a simple harness-on-a-harness that creates a set of tools and workflows on top of existing (or purpose-built) skills, turning them into what appears like your typical code review agent. It includes a command line tool that can run these skills on top of a set of changes, or even an entire codebase, as well as mechanisms to do similar within GitHub Pull Requests. The simplest way to think about it: **Warden is a framework for using LLMs to find bugs.** It is licensed under our Fair Source framework.

It does this by building on top of the Claude SDK. It wraps your skill with a meta prompt that creates a more pre-defined behavior out of it, and includes a number of other capabilities like deduplication and verification - both LLM-driven. The output ends up dual optimized for humans and machines, giving you a pleasant terminal-based interactive experience as well as a schema-bound JSONL capturing the results.



Warden Running

You can use it to enforce anything that you can connect to code, but we find it most effective at things like vulnerability detection or bug identification. If you can describe it with a skill, Warden can help you enforce it.

Results Speak

If you've been following our work here, you might remember we found a dozen Sentry-scoped vulnerabilities with the first pass of Warden some months back. None of them were critical, but it proved that leveraging inference in addition to our traditional mechanisms was going to be valuable. Since then we have been running Warden across a number of repositories at Sentry, mostly finding small bugs here and there in Pull Requests. More recently I had been wanting to experiment with another technique that seemed to be more reliable with models: scoping the goal to a much more narrow set of conditions. That meant taking our larger reaching "security vulns" skill and downgrading it to a "this specific security vuln". We combined this with a form of metaoptimization that looks quite a lot like RL - I've written about this before and I call it Synthesis - and the results are very very good.

In this pass **we found 60+ vulnerabilities in Sentry** primarily around auth/authz, including full authentication bypasses as well as a variety of indirect object references or otherwise wrongly scoped permissions that allowed legitimate destructive actions. They were all off the critical paths, and the impact was limited, but they were issues we would have absolutely paid bounties out for.

The cost of all of this? The skill itself *usually* takes $10s of dollars in token cost to build, and there's incremental refinement from there. The major costs come from backfilling and iteration - we hit just around $1,000 with opus-4.7 running the single authz skill against our backend Python code (this is the `getsentry/sentry` repo).

This time around I also wanted to see what this would look like in some other projects. I'm convinced this is a big unlock for teams, and so I figured if I could show this wasn't just Sentry it might spark some more interest. I ran this against some big open source projects that fit my criteria: highly complex and well adopted, some kind of large scoped backend service with auth, and active and old enough to almost certainly have a lot of gaps.

In Home Assistant's core with a partial scan focused on third-party components (it's a large repo!) we reported 52 findings. This took a little bit of refinement on the skill as I'm not familiar with the codebase, but I spot checked with maintainers and they validated they were real (albeit not always that important given what HASS does).

In Discourse we scanned the whole Ruby app and came out with a total of 7 findings after two fine-tuning passes. Discourse has been pretty eager on toolchain adoption and the surface area is smaller than HASS so it doesn't totally surprise me that we didn't find as much.

I won't be sharing the details of these for obvious reasons, but Home Assistant gave us permission to share an example of the outcomes.

Overall I'm very proud of what you can accomplish using this technique, and it's easy enough to teach so that's what we're here to do today.

Building an Effective Skill

Warden will take your skill and wrap it up with some syntactic sugar and a little bit of coercion, but the power is still coming from that skill. I've found the more targeted you make it the better it will perform. Within the context of security this looks like using a skill per set of security concerns rather than a generalized one. So for example, a "find-rces" vs "find-security-vulns".

To build the skill we're going to use Sentry's `skill-writer`. I have a methodology that says if something is going to change or be needed quite often, it must be done via automation or tools that are repeatable. In this case, automation is our skill-writer - a skill that just synthesizes a number of best practices and we use to simplify building and maintaining skills (including itself).

Choose a location for your skills. This can either be a global repo, in case you want these to be reusable across projects, or it can be within the project's repo itself. I recommend putting them in the `skills/` directory within that repo. I'm going to use an example of doing them in a dedicated repository.

Open your favorite coding agent and let's kick it off:

> I'm setting this folder up as a new repo which will contain skills for our warden project (warden.sentry.dev). These are going to be generalized skills (each in ./skills/skillName) which we'll use for things like bug detection, security concerns, etc. Help me get some scaffolding, docs, AGENTS.md, etc in place. You also should use dotagents.sentry.dev and setup the skill-writer skill from the getsentry/skills repo. Make the license MIT in this repository.

You may want to clean some of the repository up at this point, but you should have a basic new project setup, you should see an agents.toml with skill-writer in it, and a skills/ directory where we'll be working out of.

> In our AGENTS.md, when we build Warden skills, we should always find examples for both Python and TypeScript projects, and use them to understand and encode patterns. These skills should also be designed to trace concerns fully. We will also prefix all of these skills with 'wrdn-', and they will generally use the Read Grep Glob and Bash tools.

Now you just need to kick off synthesizing a new skill. In our case we're going to focus on authentication bypasses:

> Create a security skill that's focused on identifying authentication bypasses, permission issues, or similar high criticality concerns. Identify common frameworks and the patterns that we should be looking for. For each framework, or larger set of concerns, break them up in their own references file. Look up prior art online for best practices on identifying these concerns, what the common mistakes are. Additionally sift through the ~/src/sentry repository for similar reference material.

You may find you want to push it in a certain direction, or it needs heavier steering to get it to search online for better references. You may also find that you need to feed it a list of technologies or frameworks that matter more, or some *better* prior art online (there are both good and bad resources).

When it's done, skim through the skill to make sure it looks like what you'd expect. It's going to be a fat skill, but if it hasn't been sufficiently segmented to where it's loading information it may never need, then you should tell your agent to improve upon that (e.g. python concerns should be in a references file if you're not always evaluating python code). Often I take an active run of Warden, stop after 1-3 findings, and then feed those results back into an active agent session to validate and use the learnings to reinforce the skill.

Testing and Iterating Locally

To test this I find it's easiest to install Warden system-wide:

```
npm install -g @sentry/warden
```

Pop over to your repository where you're going to QA the skill, identify a path of critical code (I like to test on our API endpoints), and run Warden on it:

```
warden --skill ~/src/warden-skills/skills/wrdn-access-control "src/se
```

This can be both expensive and take a long time. Warden by default is going to use your Claude credentials (via the Claude SDK). You'll see it sift through a bunch of chunks across the files - it will split up one file into some smaller chunks, similar to how a pull request would look, and you might notice findings come up. All of what's happening here will get logged to a typed jsonl file that you can go back and revisit with `warden logs`. More importantly, you can feed the whole thing into your favorite coding agent, which is what we'll do next:

> I have a set of findings that I want you to verify. Use a subagent to explore each one, trace it fully, and verify the finding. The findings are available in [logfile path].

Warden Analysis

There's a handful of ways you can use the finding data, but I find having a second pass agent go over it valuable. You'll be able to dig more and fine-tune it, as it's not uncommon for Warden to get the severity wrong, or to fail to entirely trace the problem. In this case you might also find that some are false positives, this is where we can iterate. Take any false positive, dump it back into your session where you generated the skill file (or a new session if you must) and ask to evaluate what should change to improve the results. Make sure it uses `skill-writer` again if it's a fresh session.

Do this until you're satisfied, but honestly I find the better the model, the less you need to micromanage the skills (no different than prompts). I almost always find the first few sessions have false positives - either classified as worse than they are, or not deeply traced enough. Just run subjective passes til you find the accuracy is good enough. It will never be 100%, but I find you can get it to 95%+ with only a few iterations.

Just as a note, when you're reading this post, it's possible we've already shipped our broader SkillsKit (`skillet` at the time of writing). This is intended to make it easier to bundle up the act of building, improving, and evaluating (via evals) skills.

Running On Pull Requests

Warden tries to solve two concerns:

1. Finding pre-existing issues
2. Preventing them from happening in the future

The latter means running it on all forward-looking code changes, and we provide a native capability to do this via GitHub Actions. If you're not using GitHub you can certainly still instrument things, but we're big GH users ourselves.

In this scenario you can think of Warden similarly to all of the code review bots you're probably far too familiar with, except you control the focus. It will scan the PR diff, using a similar chunking strategy to what it does locally, and will annotate the PR with any findings. Additionally it will deduplicate its own findings, as well as avoid annotating anything that has already been identified by another bot. In practice, unfortunately, Warden usually finds things first and the other bots still flag their findings, but it's the thought that counts right?

Setting this up is pretty simple, and we recommend doing it organization wide at your company:

1. Go to Organization Settings → Secrets and variables → Actions
2. Add the following secrets:

   `WARDEN_ANTHROPIC_API_KEY`
   Your Anthropic API key from console.anthropic.com

   `WARDEN_MODEL` (optional)
   Override the default model (e.g. claude-opus-4-6)

`WARDEN_SENTRY_DSN` (optional)
Sentry DSN for error and performance telemetry

Now in each repository where you want to utilize Warden it's ready to go and all you have to do is configure it normally. You can do this with `npx @sentry/warden init` and it will automatically generate the baseline workflow.

We are also exploring running Warden globally via the `.github` repository, where we could enforce global skill checks only when certain things exist in a repo (like scanning for vulns in any changes to a GitHub Workflow).

Fin

I hope you try Warden, and I hope you start asking "what comes next" after we can generate slop patches. It's not dozens of agents coordinating to generate more slop patches - it's verification, detection, improving what we already can do. Every time I pick up this project to take a spin at improving it I find more active vulnerabilities in our codebases. If you take the time to try this, I am almost certain you will find them in yours too. Yes, inference is expensive, but it's cheaper than the bounties we would have paid out.

Please be responsible. You'll find our example skills on GitHub, such as the one that found a large number of Sentry vulnerabilities. If you use this for evil, human-enforced karma is a thing.

Give Warden a spin, tell me how it works, and if you find anything that you think could be better, drop a note on GitHub.

Every layer of review makes you 10x slower

AVERY PENNARUN · 26 MAR 2026 · SOURCE

2026-03-16 »

Every layer of review makes you 10x slower

We've all heard of those network effect laws: the value of a network goes up with the square of the number of members. Or the cost of communication goes up with the square of the number of members, or maybe it was n log n, or something like that, depending how you arrange the members. Anyway doubling a team doesn't double its speed; there's coordination overhead. Exactly how much overhead depends on how badly you botch the org design.

But there's one rule of thumb that someone showed me decades ago, that has stuck with me ever since, because of how annoyingly true it is. The rule is annoying because it doesn't *seem* like it should be true. There's no theoretical basis for this claim that I've ever heard. And yet, every time I look for it, there it is.

Here we go:

Every layer of approval makes a process 10x slower

I know what you're thinking. Come on, 10x? That's a lot. It's unfathomable. Surely we're exaggerating.

Nope.

Just to be clear, we're counting "wall clock time" here rather than effort. Almost all the extra time is spent sitting and waiting.

Look:

- Code a simple bug fix
  30 minutes

- Get it code reviewed by the peer next to you
  300 minutes → 5 hours → half a day

- Get a design doc approved by your architects team first
  50 hours → about a week

- Get it on some other team's calendar to do all that
  (for example, if a customer requests a feature)
  500 hours → 12 weeks → one fiscal quarter

I wish I could tell you that the next step up — 10 quarters or about 2.5 years — was too crazy to contemplate, but no. That's the life of an executive sitting above a medium-sized team; I bump into it all the time even at a relatively small company like Tailscale if I want to change product direction. (And execs sitting above large teams can't actually do work of their own at all. That's another story.)

AI can't fix this

First of all, this isn't a post about AI, because AI's direct impact on this problem is minimal. Okay, so Claude can code it in 3 minutes instead of 30? That's super, Claude, great work.

Now you either get to spend 27 minutes reviewing the code yourself in a back-and-forth loop with the AI (this is actually kinda fun); or you save 27 minutes and submit unverified code to the code reviewer, who will still take 5 hours like before, but who will now be mad that you're making them read the slop that you were too lazy to read yourself. Little of value was gained.

Now now, you say, that's not the value of agentic coding. You don't use an agent on a 30-minute fix. You use it on a monstrosity week-long project that you and Claude can now do in a couple of hours! Now we're talking. Except no, because the monstrosity is so big that your reviewer will be extra mad that you didn't read it yourself, and it's too big to review in one chunk so you have to slice it into new bite-sized chunks, each with a 5-hour review cycle. And there's no design doc so there's no intentional architecture, so eventually someone's going to push back on that and here we go with the design doc review meeting, and now your monstrosity week-long project that you did in two hours is... oh. A week, again.

I guess I could have called this post Systems Design 4 (or 5, or whatever I'm up to now, who knows, I'm writing this on a plane with no wifi) because yeah, you guessed it. It's Systems Design time again.

The only way to sustainably go faster is fewer reviews

It's funny, everyone has been predicting the Singularity for decades now. The premise is we build systems that are so smart that they themselves can build the next system that is even smarter, that builds the next smarter one, and so on, and once we get that started, if they keep getting smarter faster enough, then the incremental time (t) to achieve a unit (u) of improvement goes to zero, so (u/t) goes to infinity and foom.

Anyway, I have never believed in this theory for the simple reason we outlined above: the majority of time needed to get anything done is not actually the time doing it. It's wall clock time. Waiting. Latency.

And you can't overcome latency with brute force.

I know you want to. I know many of you now work at companies where the business model kinda depends on doing exactly that.

Sorry.

But you can't just *not review* things!

Ah, well, no, actually yeah. You really can't.

There are now many people who have seen the symptom: the start of the pipeline (AI generated code) is so much faster, but all the subsequent stages (reviews) are too slow! And so they intuit the obvious solution: stop reviewing then!

The result might be slop, but if the slop is 100x cheaper, then it only needs to deliver 1% of the value per unit and it's still a fair trade. And if your value per unit is even a mere 2% of what it used to be, you've doubled your returns! Amazing.

There are some pretty dumb assumptions underlying that theory; you can imagine them for yourself. Suffice it to say that this produces what I will call the AI Developer's Descent Into Madness:

1. Whoa, I produced this prototype so fast! I have super powers!
2. This prototype is getting buggy. I'll tell the AI to fix the bugs.
3. Hmm, every change now causes as many new bugs as it fixes.
4. Aha! But if I have an AI agent also review the code, it can find its own bugs!
5. Wait, why am I personally passing data back and forth between agents
6. I need an agent framework
7. I can have my agent write an agent framework!
8. Return to step 1

It’s actually alarming how many friends and respected peers I’ve lost to this cycle already. Claude Code only got good maybe a few months ago, so this only recenlty started happening, so I assume they will emerge from the spiral eventually. I mean, I hope they will. We have no way of knowing.

Why we review

Anyway we know our symptom: the pipeline gets jammed up because of too much new code spewed into it at step 1. But what's the root cause of the clog? Why doesn’t the pipeline go faster?

I said above that this isn’t an article about AI. Clearly I’m failing at that so far, but let’s bring it back to humans. It goes back to the annoyingly true observation I started with: every layer of review is 10x slower. As a society, we know this. Maybe you haven't seen it before now. But trust me: people who do org design for a living know that layers are expensive... and they still do it.

As companies grow, they all end up with more and more layers of collaboration, review, and management. Why? Because otherwise mistakes get made, and mistakes are increasingly expensive at scale. The average value added by a new feature eventually becomes lower than the average value lost through the new bugs it causes. So, lacking a way to make features produce more value (wouldn't that be nice!), we try to at least reduce the damage.

The more checks and controls we put in place, the slower we go, but the more monotonically the quality increases. And isn’t that the basis of *continuous improvement?*

Well, sort of. Monotonically increasing quality is on the right track. But “more checks and controls” went off the rails. That’s *only one way* to improve quality, and it's a fraught one.

“Quality Assurance” reduces quality

I wrote a few years ago about W. E. Deming and the "new" philosophy around quality that he popularized in Japanese auto manufacturing. (Eventually U.S. auto manufacturers more or less got the idea. So far the software industry hasn’t.)

One of the effects he highlighted was the problem of a “QA” pass in a factory: build widgets, have an inspection/QA phase, reject widgets that fail QA. Of course, your inspectors probably miss some of the failures, so when in doubt, add a second QA phase after the first to catch the remaining ones, and so on.

In a simplistic mathematical model this seems to make sense. (For example, if every QA pass catches 90% of defects, then after two QA passes you've reduced the number of defects by 100x. How awesome is that?)

But in the reality of agentic humans, it's not so simple. First of all, the incentives get weird. The second QA team basically serves to evaluate how well the first QA team is doing; if the first QA team keeps missing defects, fire them. Now, that second QA team has little incentive to produce that outcome for their friends. So maybe they don't look too hard; after all, the first QA team missed the defect, it's not unreasonable that we might miss it too.

Furthermore, the first QA team knows there is a second QA team to catch any defects; if I don't work too hard today, surely the second team will pick up the slack. That's why they're there!

Also, the team making the widgets in the first place doesn't check their work too carefully; that's what the QA team is for! Why would I slow down the production of every widget by being careful, at a cost of say 20% more time, when there are only 10 defects in 100 and I can just eliminate them at the next step for only a 10% waste overhead? It only makes sense. Plus they'll fire me if I go 20% slower.

To say nothing of a whole engineering redesign to improve quality, that would be super expensive and we could be designing all new widgets instead.

Sound like any engineering departments you know?

Well, this isn't the right time to rehash Deming, but suffice it to say, he was on to something. And his techniques worked. You get things like the famous Toyota Production System where they eliminated the QA phase entirely, but gave everybody an "oh crap, stop the line, I found a defect!" button.

Famously, US auto manufacturers tried to adopt the same system by installing the same "stop the line" buttons. Of course, nobody pushed those buttons. They were afraid of getting fired.

Trust

The basis of the Japanese system that worked, and the missing part of the American system that didn't, is trust. Trust among individuals that your boss Really Truly Actually wants to know about every defect, and wants you to stop the line when you find one. Trust among managers that executives were serious about quality. Trust among executives that individuals, given a system that can work and has the right incentives, will produce quality work and spot their own defects, and push the stop button when they need to push it.

But, one more thing: trust that the system *actually does work*. So first you need a system that will work.

Fallibility

AI coders are fallible; they write bad code, often. In this way, they are just like human programmers.

Deming's approach to manufacturing didn't have any magic bullets. Alas, you can't just follow his ten-step process and immediately get higher quality engineering. The secret is, you have to get your engineers to engineer higher quality into the whole system, from top to bottom, repeatedly. Continuously.

Every time something goes wrong, you have to ask, "How did this happen?" and then do a whole post-mortem and the Five Whys (or however many Whys are in fashion nowadays) and fix the underlying Root Causes so that it doesn't happen again. "The coder did it wrong" is never a root cause, only a symptom. Why was it possible for the coder to get it wrong?

The job of a code reviewer isn't to review code. It's to figure out how to obsolete their code review comment, that whole class of comment, in all future cases, until you don't need their reviews at all anymore.

(Think of the people who first created "go fmt" and how many stupid code review comments about whitespace are gone forever. Now that's engineering.)

By the time your review catches a mistake, the mistake has already been made. The root cause happened already. You're too late.

Modularity

I wish I could tell you I had all the answers. Actually I don't have much. If I did, I'd be first in line for the Singularity because it sounds kind of awesome.

I think we're going to be stuck with these systems pipeline problems for a long time. Review pipelines — layers of QA — don't work. Instead, they make you slower while hiding root causes. Hiding causes makes them harder to fix.

But, the call of AI coding is strong. That first, fast step in the pipeline is *so fast!* It really does feel like having super powers. I want more super powers. What are we going to do about it?

Maybe we finally have a compelling enough excuse to fix the 20 years of problems hidden by code review culture, and replace it with a real culture of quality.

I think the optimists have half of the right idea. Reducing review stages, even to an uncomfortable degree, is going to be needed. But you can't just reduce review stages without something to replace them. That way lies the Ford Pinto or any recent Boeing aircraft.

The complete package, the table flip, was what Deming brought to manufacturing. You can't half-adopt a "total quality" system. You need to eliminate the reviews *and* obsolete them, in one step.

How? You can fully adopt the new system, in small bites. What if some components of your system can be built the new way? Imagine an old-school U.S. auto manufacturer buying parts from Japanese suppliers; wow, these parts are so well made! Now I can start removing QA steps elsewhere because I can just assume the parts are going to work, and my job of "assemble a bigger widget from the parts" has a ton of its complexity removed.

I like this view. I've always liked small beautiful things, that's my own bias. But, you can assemble big beautiful things from small beautiful things.

It's a lot easier to build those individual beautiful things in small teams that trust each other, that know what quality looks like *to them*. They deliver their things to customer teams who can clearly explain what quality looks like *to them*. And on we go. Quality starts bottom-up, and spreads.

I think small startups are going to do really well in this new world, probably better than ever. Startups already have fewer layers of review just because they have fewer people. Some startups will figure out how to produce high quality components quickly; others won't and will fail. Quality by natural selection?

Bigger companies are gonna have a harder time, because their slow review systems are baked in, and deleting them would cause complete chaos.

But, it's not just about company size. I think engineering teams at any company can get smaller, and have better defined interfaces between them.

Maybe you could have multiple teams inside a company competing to deliver the same component. Each one is just a few people and a few coding bots. Try it 100 ways and see who comes up with the best one. Again, quality by evolution. Code is cheap but good ideas are not. But now you can try out new ideas faster than ever.

Maybe we'll see a new optimal point on the monoliths-microservices continuum. Microservices got a bad name because they were too micro; in the original terminology, a "micro" service was exactly the right size for a "two pizza team" to build and operate on their own. With AI, maybe it's one pizza and some tokens.

What's fun is you can also use this new, faster coding to experiment with different module *boundaries* faster. *Features* are still hard for lots of reasons, but refactoring and automated integration testing are things the AIs excel at. Try splitting out a module you were afraid to split out before. Maybe it'll add some lines of code. But suddenly lines of code are cheap, compared to the coordination overhead of a bigger team maintaining both parts.

Every team has some monoliths that are a little too big, and too many layers of reviews. Maybe we won't get all the way to Singularity. But, we can engineer a much better world. Our problems are solvable.

It just takes trust.

Related

Highlights on "quality," and Deming's work as it applies to software development (2016)

Systems design 3: LLMs and the semantic revolution (2025)

Unrelated

SimSWE part 2: The perils of multitasking (2017)

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · SOURCE

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this opinion piece by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into AI psychosis. Here's a representative paragraph from the piece:

> In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training[1].

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess[2]. AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there[3].

When AI people talk about LLMs having personalities, or wanting things, or even having souls[4], these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a recent paper about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

   ↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

   ↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

   ↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

   ↩

Anthropic appoints KiYoung Choi as Representative Director of Korea ahead of Seoul office opening

ANTHROPIC NEWS · 27 MAY 2026 · SOURCE

Korea is one of the most active markets in the world for Claude.ai. According to our latest Economic Index, Koreans use Claude at more than 3.5 times the rate expected for the population size, with usage skewing heavily toward technical and creative work. To support that momentum, KiYoung Choi is joining Anthropic as Representative Director of Korea, ahead of the opening of our Seoul office. In the coming weeks, senior leadership from Anthropic will travel to Seoul to officially open the office and meet with customers.

KiYoung joins from Snowflake, where he served as General Manager for Korea. He brings over three decades of experience leading technology businesses across Korea and Asia-Pacific. Throughout his career, he has helped some of the largest Korean enterprises navigate transformative shifts in technology, ranging from cloud computing to AI adoption. Previously, he held country leadership roles at Google Cloud, Adobe, Autodesk, and Microsoft.

“Korea is one of the most sophisticated AI markets in the world, leading in hardware innovation, developer activity, and enterprise adoption,” KiYoung said. “Korean organizations combine technical depth with a commitment to responsible deployment, which is exactly where Anthropic operates. That alignment is what brought me to Anthropic—and why we are focused on building in Korea for the long term.”

At Anthropic, KiYoung will lead a go-to-market strategy that supports the unique uses of Claude in Korea. Innovative startups and leading enterprises in Korea are already building with Claude. Law&Company uses Claude to power its AI legal assistant, helping lawyers reduce time spent on legal research and document preparation while maintaining high accuracy for sensitive legal

work. SK Telecom, the largest telecommunications company in Korea, chose Claude to build a custom AI customer service model, helping improve service quality and support customer service teams at SKT.

“Korea is one of the markets where we’ve seen the most enthusiasm for Claude, and few people understand its technology landscape the way KiYoung does,” said Chris Ciauri, Managing Director of International, Anthropic. “He’ll build the team and the local partnerships to support how Korean organizations are putting Claude to work.”

Our Korea team will focus on building partnerships with enterprises and startups, engaging with government and research institutions, and supporting the vibrant developer community building with Claude.

For more information about career opportunities in our Seoul office, visit anthropic.com/careers.